ASSIGNMENT 2 - VS CODE QUESTIONS

Avni Sharma

Functions – Question 1: Find maximum of three integers

Write `int maxOfThree(int a, int b, int c)` and return the largest value.

```
#include <stdio.h>

int maxOfThree(int a, int b, int c) {
    int max = a;
    if (b > max) max = b;
    if (c > max) max = c;
    return max;
}

int main(void) {
    int a, b, c;
    scanf("%d %d %d", &a, &b, &c);
    printf("%d\n", maxOfThree(a, b, c));
    return 0;
}
```

Functions – Question 2: Count multiples in a range

Write `countMultiples(n,k)` to count positive integers from 1 through n divisible by positive k.

```
#include <stdio.h>

long long countMultiples(long long n, long long k) {
    if (n < 1 || k < 1) return 0;
    return n / k;
}

int main(void) {
    long long n, k;
    scanf("%lld %lld", &n, &k);
    printf("%lld\n", countMultiples(n, k));
    return 0;
}
```

Functions – Question 3: Rectangle area function

Write a function that returns the area of a rectangle from length and width; print the return value in main.

```
#include <stdio.h>

long long rectangleArea(long long length, long long width) {
    return length * width;
}

int main(void) {
    long long length, width;
    scanf("%lld %lld", &length, &width);
    printf("%lld\n", rectangleArea(length, width));
    return 0;
}
```

Functions – Question 4: Perfect square without sqrt

Return 1 if a non-negative integer is a perfect square, otherwise 0. Do not use floating-point square root.

```
#include <stdio.h>

int isPerfectSquare(long long n) {
    if (n < 0) return 0;
    long long lo = 0, hi = 3037000499LL;   /* floor(sqrt(2^63 - 1)) */
    while (lo <= hi) {
        long long mid = lo + (hi - lo) / 2;
        long long sq = mid * mid;
        if (sq == n) return 1;
        if (sq < n) lo = mid + 1;
        else        hi = mid - 1;
    }
    return 0;
}

int main(void) {
    long long n;
    scanf("%lld", &n);
    printf("%d\n", isPerfectSquare(n));
    return 0;
}
```

Functions – Question 5: GCD using Euclid's algorithm

Implement `gcd(a,b)` and print the greatest common divisor. Inputs are not both zero.

```
#include <stdio.h>

long long gcd(long long a, long long b) {
    if (a < 0) a = -a;
    if (b < 0) b = -b;
    while (b != 0) {
        long long t = a % b;
        a = b;
        b = t;
    }
    return a;
}

int main(void) {
    long long a, b;
    scanf("%lld %lld", &a, &b);
    printf("%lld\n", gcd(a, b));
    return 0;
}
```

Functions – Question 6: Convert seconds

Convert total seconds into hours, remaining minutes, and remaining seconds.

```
#include <stdio.h>

int main(void) {
    long long total;
    scanf("%lld", &total);

    long long hours   = total / 3600;
    long long minutes = (total % 3600) / 60;
    long long seconds = total % 60;

    printf("%lld %lld %lld\n", hours, minutes, seconds);
    return 0;
}
```

Functions – Question 7: Evaluate a piecewise function

Compute f(x): x*x+2*x+1 if x<0; 3*x+2 if 0<=x<10; 5*x-7 if x>=10.

```
#include <stdio.h>

long long f(long long x) {
    if (x < 0)  return x * x + 2 * x + 1;
    if (x < 10) return 3 * x + 2;
    return 5 * x - 7;
}

int main(void) {
    long long x;
    scanf("%lld", &x);
    printf("%lld\n", f(x));
    return 0;
}
```

Nested loops – Question 1: Count pairs with a target sum

Read n values and count index pairs i<j whose values sum to target.

```
#include <stdio.h>

int main(void) {
    int n;
    long long target, a[1000];
    int count = 0;

    scanf("%d %lld", &n, &target);
    for (int i = 0; i < n; i++)
        scanf("%lld", &a[i]);

    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            if (a[i] + a[j] == target)
                count++;

    printf("%d\n", count);
    return 0;
}
```

Nested loops – Question 2: Count relatively prime values

Count x in [1,m] for which gcd(x,n)=1. Use a gcd helper function.

```
#include <stdio.h>

long long gcd(long long a, long long b) {
    while (b != 0) {
        long long t = a % b;
        a = b;
        b = t;
    }
    return a;
}

int main(void) {
    long long n, m;
    int count = 0;
    scanf("%lld %lld", &n, &m);

    for (long long x = 1; x <= m; x++)
        if (gcd(x, n) == 1)
            count++;

    printf("%d\n", count);
    return 0;
}
```

Nested loops – Question 3: Pythagorean triples up to N

List triples 1<=a<b<c<=N satisfying $a^2+b^2=c^2$, ordered by a then b then c. Print None if absent.

```
#include <stdio.h>

int main(void) {
    long long N;
    int found = 0;
    scanf("%lld", &N);

    for (long long a = 1; a <= N; a++)
        for (long long b = a + 1; b <= N; b++)
            for (long long c = b + 1; c <= N; c++)
                if (a * a + b * b == c * c) {
                    printf("%lld %lld %lld\n", a, b, c);
                    found = 1;
                }

    if (!found) printf("None\n");
    return 0;
}
```

Nested loops – Question 4: Pairs with equal digit sums

Count index pairs i<j whose values have equal digit sums.

```
#include <stdio.h>

int digitSum(long long x) {
    int sum = 0;
    while (x > 0) {
        sum += x % 10;
        x /= 10;
    }
    return sum;
}

int main(void) {
    int n, count = 0;
    long long a[1000];

    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        scanf("%lld", &a[i]);

    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            if (digitSum(a[i]) == digitSum(a[j]))
                count++;

    printf("%d\n", count);
    return 0;
}
```

Nested loops – Question 5: Closest pair difference

Find the minimum absolute difference between any two distinct input values using nested loops.

```
#include <stdio.h>

int main(void) {
    int n;
    long long a[1000];

    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        scanf("%lld", &a[i]);

    long long best = -1;
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            long long d = a[i] - a[j];
            if (d < 0) d = -d;
            if (best == -1 || d < best)
                best = d;
        }
    }

    printf("%lld\n", best);
    return 0;
}
```

1D arrays – Question 1: Longest increasing contiguous run

Find the length of the longest contiguous strictly increasing subarray.

```
#include <stdio.h>

int main(void) {
    int n;
    long long a[100000];

    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        scanf("%lld", &a[i]);

    if (n == 0) {
        printf("0\n");
        return 0;
    }

    int best = 1, current = 1;
    for (int i = 1; i < n; i++) {
        if (a[i] > a[i - 1])
            current++;
        else
            current = 1;
        if (current > best)
            best = current;
    }

    printf("%d\n", best);
    return 0;
}
```

Strings and character arrays – Question 1: Count vowels, consonants, digits, spaces

Read a line and count English vowels, consonants, digits, and spaces; ignore punctuation.

```
#include <stdio.h>
#include <ctype.h>

int main(void) {
    char line[1000];
    int vowels = 0, consonants = 0, digits = 0, spaces = 0;

    fgets(line, sizeof(line), stdin);

    for (int i = 0; line[i] != '\0'; i++) {
        char ch = line[i];
        if (isalpha((unsigned char)ch)) {
            char lower = tolower((unsigned char)ch);
            if (lower == 'a' || lower == 'e' || lower == 'i' ||
                lower == 'o' || lower == 'u')
                vowels++;
            else
                consonants++;
        } else if (isdigit((unsigned char)ch)) {
            digits++;
        } else if (ch == ' ') {
            spaces++;
        }
    }

    printf("%d %d %d %d\n", vowels, consonants, digits, spaces);
    return 0;
}
```

Strings and character arrays – Question 2: Run-length encode

Compress consecutive repeated letters as character followed by run length; e.g. aaabbc -> a3b2c1.

```
#include <stdio.h>

int main(void) {
    char s[1000];
    scanf("%999s", s);

    int i = 0;
    while (s[i] != '\0') {
        char ch = s[i];
        int run = 0;
        while (s[i] == ch) {
            run++;
            i++;
        }
        printf("%c%d", ch, run);
    }
    printf("\n");
    return 0;
}
```

2D arrays – Question 1: Row and column sums

Print all row sums on line one and column sums on line two.

```
#include <stdio.h>

int main(void) {
    int r, c;
    long long m[100][100];
    long long rowSum[100] = {0}, colSum[100] = {0};

    scanf("%d %d", &r, &c);
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) {
            scanf("%lld", &m[i][j]);
            rowSum[i] += m[i][j];
            colSum[j] += m[i][j];
        }
    }

    for (int i = 0; i < r; i++)
        printf("%lld%c", rowSum[i], i == r - 1 ? '\n' : ' ');
    for (int j = 0; j < c; j++)
        printf("%lld%c", colSum[j], j == c - 1 ? '\n' : ' ');

    return 0;
}
```

2D arrays – Question 2: Check matrix symmetry

For a square matrix, test whether matrix[i][j] equals matrix[j][i] for all i,j.

```
#include <stdio.h>

int main(void) {
    int n, symmetric = 1;
    long long m[100][100];

    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%lld", &m[i][j]);

    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            if (m[i][j] != m[j][i])
                symmetric = 0;

    printf("%s\n", symmetric ? "YES" : "NO");
    return 0;
}
```